Diabetic Retinopathy Detection and Classification using Deep learning

A Major Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

in

Computer Science and Engineering

By

PIDUGUSAI PRASANNA	**2456-22-733-114**
BOKINALA RESHMITHA	**2456-22-733-076**
KYATHAM PRANATHI	**2456-22-733-309**

Under the guidance of

Mrs S PALLAVI

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Gokaraju Lailavathi Engineering College

(Affiliated to Osmania University)

Nizampet, Hyderabad-500090

(2025-2026)

Department of Computer Science and Engineering

Gokaraju Lailavathi Engineering College

(Affiliated to Osmania University)

Nizampet, Hyderabad-500090

(2025-2026)

CERTIFICATE

This is to Certify that A Major Project report entitled **"Diabetic Retinopathy Detection and Classification using Deep learning** " is being submitted by Pidugu Sai Prasanna **(**2456-22-733-114**),** Bokinala Reshmitha **(**2456-22-733-076**),** Kyatham Pranathi (2456-22-733-309) in partial fulfillment of the requirement of the award for the degree of Bachelor of Engineering in "Computer Science and Engineering" O.U. Hyderabad during the year 2025-2026 is a record of bonafide work carried out by them under my guidance. The results presented in this project have been verified and are found to be satisfactory.

Project Guide

Mrs S Pallavi

Assistant Professor

H.O.D

Dr. Padmalaya Nayak

Professor & Head,

Dept. of CSE

External Examiner(s)

Department of Computer Science and Engineering

Gokaraju Lailavathi Engineering College

(Affiliated to Osmania University)

Nizampet, Hyderabad-500090

(2025-2026)

DECLARATION

We, Pidugu Sai Prasanna Ht.No. 2456-22-733-114**,** Bokinala Reshmitha Ht.No 2456-22-733-076**,** Kyatham Pranathi Ht.No. 2456-22-733-309 hereby certify that the major project entitled "**Diabetic Retinopathy Detection and Classification using Deep learning**" is submitted in the partial fulfilment of the required for the award of the degree of Bachelor of Engineering in Computer Science and Engineering.

This is a record work carried out by us under the guidance of Mrs S Pallavi,Assistant Professor, Gokaraju Lailavathi Engineering College, Nizampet.. The results embodied in this report have not been reproduced/copied from any source. The results embodied in this report have not been submitted to any other university or institute for the award of any other degree or diploma.

PIDUGU SAI PRASANNA	**2456-22-733-149**
BOKINALA RESHMITHA	**2456-22-733-076**
KYATHAM PRANATHI	**2456-22-733-309**

ACKNOWLEDGEMENT

It is our privilege and pleasure to express my profound sense of respect, gratitude and indebtedness to our guide Mrs.S Pallavi, Assistant Professor, Department of Computer Science and Engineering (CSE), Gokaraju Lailavathi Engineering College,, for her inspiration, guidance, cogent discussion, constructive criticisms and encouragement throughout this dissertation work.

We express our sincere thanks to Project Coordinator Mr Nandha Kishore, Assistant Professor, Department of Computer Science and Engineering, Gokaraju Lailavathi Engineering College, for her valuable suggestions and constant helping in completing the work.

We express our sincere gratitude to Dr. Padmalaya Nayak, Professor & Head of the Department of Computer Science and Engineering, Gokaraju Lailavathi Engineering College, for her precious suggestions , motivation and co-operation.

We express our sincere thanks to Dr. A. Sai Hanuman, Principal, Gokaraju Lailavathi Engineering College, Nizampet, Hyderabad, for his encouragement and constant help.

We extend our sincere thanks to all the teaching and non-teaching staff of Computer Science and Engineering Department for their support and encouragement.

Last but not least, we wish to acknowledge my friends and family members for giving moral strength moral strength and helping us to complete this dissertation.

ABSTRACT

Diabetic Retinopathy (DR) is a vision-threatening complication of diabetes and one of the leading causes of blindness worldwide. Early detection is critical to prevent irreversible damage, but manual screening is labor-intensive and subject to variability. This project proposes an automated, machine learning-based approach for accurate detection and classification of DR severity using retinal fundus images.

The system architecture begins with image retrieval and preprocessing steps such as data cleaning, normalization, and augmentation to handle class imbalance and improve generalization. Data augmentation techniques like rotation, flipping, resizing, and color jitter are applied to underrepresented classes, while oversampled classes are down-sampled to maintain uniformity across categories.

The processed dataset is split into training and validation sets, and a deep convolutional neural network, Inception v3, pre-trained on ImageNet, is fine-tuned for DR classification. The model is optimized using a weighted cross-entropy loss function to address class imbalance, and trained with the AdamW optimizer and a learning rate scheduler for efficient convergence. Model performance is evaluated using a wide range of metrics including accuracy, precision, recall, F1-score, specificity, false positive rate (FPR), false negative rate (FNR), mean squared error (MSE), and Area Under the Curve (AUC).

Experimental results demonstrate the model's effectiveness in accurately predicting DR severity levels ranging from 0 (No DR) to 4 (Proliferative DR), showcasing the potential of deep learning and machine learning techniques in supporting early diagnosis and large-scale diabetic eye screening programs.

Keywords:

Diabetic Retinopathy, Inception v3, Deep Learning, Medical Image Classification, Data Augmentation, Class Imbalance, Deep Learning Approach, Retina Image Analysis, PyTorch, Image Preprocessing, Performance Metrics.

CONTENTS

Chapter-1 INTRODUCTION.....1

1.1 MOTIVATION.....1

1.2 PROBLEM STATEMENT.....1

1.3 PROJECT OBJECTIVE.....1

Chapter-2 LITERATURE SURVEY 3

2.1 MACHINE LEARNING PIPELINE FOR DR DETECTION.....3

2.2 USE OF PRE-TRAINED MODULES: INCEPTION V3 FOR FUTURE REPRESENTATION….....4

2.3 INCEPTION V3: ARCHITECTURE OVERVIEW 4

2.4 INCEPTION V3 ADVANTAGES FOR DR CLASSIFICATION 5

2.5 INTEGRATION OF ADDITIONAL FEATURE.....5

2.6 DEPLOYMENT AND REAL-WORLD USE.....5

2.7 EXISTING SYSTEMS.....6

2.8 LIMITATIONS OF EXISITNG SYSTEM 6

Chapter-3 SOFTWARE REQUIREMENT SPECIFICATION 7

3.1 SOFTWARE REQUIREMENTS 7

3.2 HARDWARE REQUIREMENTS 7

3.3 FUNTIONAL REQUIREMENTS 8

3.4 NON FUNTIONAL REQUIREMENTS.....9

Chapter-4 SYSTEM DESIGN…..... 10

4.1 SYSTEM DESIGN.....10

4.2 SYSTEM ARCHITECTURE 11

4.3 PROPOSED METHODOLOGY… 12

4.4 UML DESIGN 13

4.4.1 CLASS DIAGRAM..... 16

4.4.2 USECASE DIAGRAM 16

4.4.3 COMPONENT DIAGRAM 17

4.4.4 SEQUENCE DIAGRAM 19

4.4.5 ACTIVITY DIAGRAM..... 20

4.5 TECHNOLOGY DESCRIPTION..... 21

Chapter-5 IMPLEMENTATION..... 24

5.1 ENVIRONMENT SETUP..... 24

5.2 MODULES..... 26

5.3 EXECUTABLE CODE 27

Chapter-6 TESTING… 43

6.1 TESTING DEFINATION… 43

6.2 UNIT TESTING..... 43

Chapter-7 RESULTS 46

Chapter-8 CONCLUSION 51

8.1 CONCLUSION..... 51

8.2 FUTURE SCOPE..... 51

REFERENCES 53

SNO.	LIST OF FIGURES	PAGE NO.
2.3.1	Architecture of Inception v3	5
4.2.1	System Architecture	11
4.3.1	Proposed Methodology	12
4.4.1	UML Heirarchy Diagram	15
4.4.1	Class Diagram	16
4.4.2	Use case Diagram	17
4.4.3	Sequence Diagram	18
4.4.4	Component Diagram	19
4.4.5	Activity Diagram	20
4.4.2	Data set	21
7.1	Index page	46
7.2	Image selection page	46
7.3	Analyze page	47
7.4	Image for No DR	47
7.5	Image for mild	48
7.6	Image for moderate	48
7.7	Image for severe	49
7.8	Image for proliferative DR	50

CHAPTER-1

INTRODUCTION

1.1 MOTIVATION

Diabetic Retinopathy (DR) is a leading cause of blindness among diabetic individuals, particularly when left undiagnosed in its early stages. Traditional screening methods rely on manual inspection of retinal fundus images by medical experts, which is time-consuming, costly, and often inaccessible in remote or underserved areas. To overcome these challenges, this project leverages deep learning techniques to build an automated and efficient DR detection system. Deep learning enables the model to learn complex patterns directly from image data, improving diagnostic accuracy and scalability. By automating the screening process, this approach aims to assist healthcare providers in early identification and intervention, ultimately reducing the risk of vision loss.

1.2 PROBLEM STATEMENT

Diabetic Retinopathy (DR) is a progressive eye disease caused by complications of diabetes and is a leading cause of preventable blindness. Early detection is crucial for effective treatment, yet traditional diagnosis through manual examination of retinal fundus images is time-consuming, prone to human error, and inaccessible in many under-resourced areas. Additionally, medical image datasets often suffer from class imbalance and quality variation, making automated detection challenging. To address these issues, there is a need for an intelligent, automated system that leverages both machine learning and deep learning techniques to accurately detect and classify the severity of DR. Such a system can enhance screening efficiency, support ophthalmologists, and enable timely interventions to reduce vision loss risks.

1.3 PROJECT OBJECTIVE

The primary objective of this project is to design and develop a highly accurate and automated machine learning framework for the detection and classification of Diabetic Retinopathy (DR) using retinal fundus images. The system will leverage advanced deep learning models, particularly Convolutional Neural Networks (CNNs), enhanced through transfer learning, to

effectively capture complex patterns and features associated with different stages of DR—No DR, Mild, Moderate, Severe, and Proliferative DR. To improve the quality of input data, the project aims to implement robust image preprocessing techniques including noise reduction, normalization, and image enhancement. This will enable better feature extraction and, consequently, higher classification accuracy. The framework will be trained and tested on publicly available datasets, ensuring generalizability and scalability to real-world clinical settings.

An important aspect of the project is the integration of explainable AI methods such as Grad-CAM to visualize and highlight the regions of retinal images influencing the model's predictions. This will enhance the transparency of the system and foster greater trust and acceptance among healthcare professionals.Finally, the project will rigorously evaluate model performance using comprehensive metrics like accuracy, precision, recall, F1-score, and AUC-ROC, benchmarking against existing methods. The ultimate goal is to provide a clinically viable, efficient, and interpretable tool that aids ophthalmologists in early diagnosis, facilitates timely treatment decisions, and helps reduce the risk of vision loss among diabetic patients (Shamrat., 2023).

CHAPTER 2
LITERATURE SURVEY

Diabetic Retinopathy Classification and Detection Using Machine Learning

Machine learning has been a promising strategy for detecting and classifying Diabetic Retinopathy (DR) to counter the increasing prevalence of diabetes-caused vision impairment. Manual examination by experts of retinal fundus images is generally time-consuming, subject to errors, and resource-constrained, especially in remote areas. DR screening operations can be automated with the help of machine learning methods, thus facilitating large-scale uniform and timely diagnosis.

2.1 Machine Learning Pipeline for DR Detection

A machine learning pipeline for the classification of DR consists of the following key steps:

1. **Image Preprocessing:** This consists of resizing, color normalization, contrast adjustment, and noise filtering of fundus images. These processes normalize the input data and make it appropriate for feature analysis (Aishwarya., 2023).
2. **Feature Engineering**: Rather than manually examining images, machine learning algorithms depend on measurable features like:

- Color histograms
- Texture statistics (e.g., pixel intensity standard deviation)
- Morphological characteristics (e.g., lesion numbers, vessel patterns) (Shamrat., 2023)

3. Dimensionality Reduction: Methods such as feature selection or Principal Component Analysis (PCA) assist in the removal of redundant features and increase model efficiency (suedumrong.,2021).

4. Model Training: Conventional machine learning classifiers such as SVM, Random Forests, and k-Nearest Neighbors (k-NN) are trained using labeled data to learn mapping patterns between DR severity levels and features.

5. Evaluation: Models are tested on unseen test data based on performance metrics like accuracy, precision, recall, F1-score, and AUC-ROC for measuring clinical feasibility.

2.2 Use of Pre-Trained Models: InceptionV3 for Feature Representation

In contrast to traditional models, which depended extensively on handcrafted features, more recent developments utilize deep feature extractors to enhance prediction performance. One of the best performing architectures is InceptionV3, invented by Google, that extracts rich, discriminative image features using several parallel convolutions.

InceptionV3 is not employed for end-to-end classification in our work. Rather, it serves as a feature extractor. The deep features from its second last layer are fed into a traditional machine learning classifier for making the final decision, leveraging deep learning's representation capability as well as traditional ML's explainability (Radhika.,2022).

2.3 InceptionV3: Architecture Overview

InceptionV3 is a sophisticated model in the Inception series aimed at extracting discriminative image features with high efficiency and low computational cost. Its distinctive architecture contains Inception Modules that execute numerous operations in parallel and then concatenate the output. This enables the model to learn both global and local patterns adequately.

Main Elements of InceptionV3 Architecture:

1.Stem Layers:

Early convolution and pooling operations decrease image sizes and ready inputs for further processing.

2.Factorized Convolutions:

Rather than relying on enormous convolutions (e.g., 5×5), InceptionV3 decomposes them into smaller ones (e.g., 1×5 and 5×1), cutting down on parameters and speed.

3.Inception Modules:

Each module carries out a series of operations (1×1, 3×3, 5×5 convolutions, and pooling) in parallel. Their outputs are concatenated, allowing multi-scale feature analysis within a single block.

4.Auxiliary Classifiers:

These intermediate layers aid training by supplying supplementary gradient signals and regularizing the network.

5.Global Average Pooling (GAP):

Replaces dense layers to minimize overfitting by averaging feature maps.

6.Output Layer:

Outputs probability distribution over classes (e.g., DR stages), often with a softmax activation.

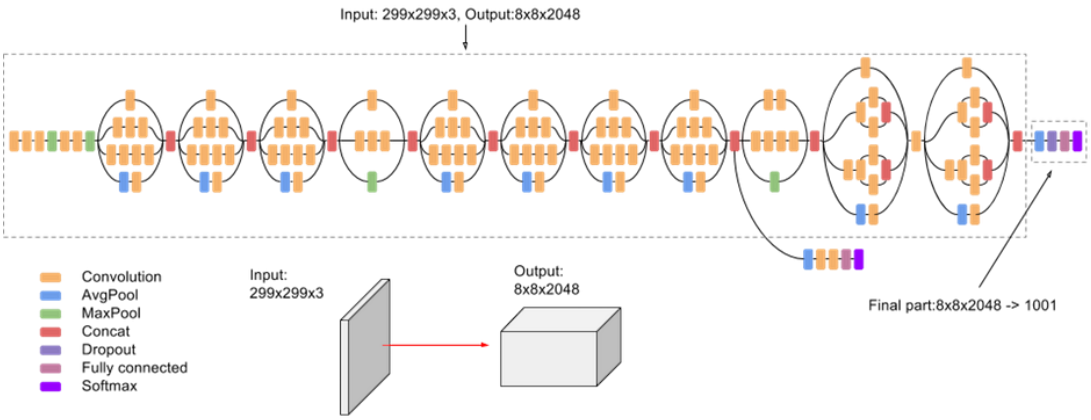


Fig 2.3.1: Architecture of Inception v3

2.4 InceptionV3 Advantages for DR Classification

1. Multi-resolution feature extraction allows detection of small, fine-grained lesions (e.g.microaneurysms) and larger abnormalities (e.g., hemorrhages).
2. Decreased computation via factorized convolutions makes it appropriate for large image datasets.
3. Increased generalization with auxiliary classifiers and regularization techniques.
4. Transfer learning assistance, so that it can be applied again with DR datasets even when initially trained on generic images such as ImageNet.

2.5 Integration of Additional Features

In reality, system performance can be enhanced further by adding non-image features. These are:

- Patient metadata (age, diabetes duration)
- Historical clinical data
- Lesion-type-specific segmentation masks
- Multi-modal inputs integrating image and structured EHR information

This hybrid model improves prediction quality by mimicking the global evaluation performed by human clinicians.

2.6 Deployment and Real-World Use

After training and validation, the system is deployed via a cloud-based API. Users upload fundus images via a web interface. The image is preprocessed, InceptionV3 is used to extract features, and the ultimate prediction is obtained from a machine learning classifier.

Deployment aspects include:

- Flask-based API
- MongoDB database to store image logs and user data
- React front-end for image upload and result display
- They enable real-time DR screening in clinics and outlying areas, assisting ophthalmologists in prioritizing patients and facilitating early treatment.

2.7 Existing System

Today, in the healthcare system, Diabetic Retinopathy screening relies predominantly on manual screening from ophthalmologists, which is subjective and time-consuming. Patients are subjected to fundus photography, and specialists visually grade the stage of DR. But with the rising number of diabetic patients, the process has turned out to be inefficient and hard to scale. Thus, there is a need for an automated system that is easy to use and allows early detection, grading, and decision support for clinicians.

2.8 Limitations of Existing System

1. Time-intensive manual processes render it challenging to treat large numbers effectively.
2. Expert ophthalmologists limitation restricts accessibility to rural and underserved areas.
3. Lack of automation and integration results in slower turnaround and delayed diagnosis.
4. Inconsistent diagnosis due to human subjectivity can affect clinical decisions.

CHAPTER-3

SOFTWARE REQUIREMENT SPECIFICATION

This chapter gives an overview of the software and hardware components required for our project.

3.1 SOFTWARE REQUIREMENTS

- **Coding Language**: Python
- **Web Framework**: Flask
- **ORM Tool**: Flask-SQLAlchemy
- **Database**: SQLite, MySQL, PostgreSQL
- **Machine Learning Framework**: PyTorch, Torchvision
- **Data Handling Libraries**: NumPy, Scikit-learn
- **Image Processing Library**: PIL
- **Data Visualization Tools**: Matplotlib, Seaborn
- **Model Architecture**: Inception v3 with auxiliary classifier
- **Development Environment**: Jupyter Notebook, Google Colab
- **IDE / Editor**: Visual Studio Code, PyCharm

3.2 HARDWARE REQUIREMENTS

- **Processor:** Intel(R) Core(TM) i3-10110U CPU @ 2.10GHz 2.59GHz or higher
- **Installed RAM:** 8.00 GB or more
- **Operating System:** 64-bit OS, x64-based processor
- **I/O Devices:** Keyboard, Mouse, Monitor
- **Storage:** Minimum 10 GB free space (for datasets, model checkpoints, and logs)

3.3 FUNCTIONAL REQUIREMENTS

These are the core capabilities the system must provide as per project goals. Each of these functionalities is essential for successful DR detection and classification.

- Image Data Collection
- Image Preprocessing and Augmentation
- Dataset Label Encoding
- Dataset Splitting (Training & Validation Sets)
- Deep Learning Model Selection (Inception v3 or other CNNs
- Model Training
- Model Validation and Testing
- Performance Metrics Evaluation (Accuracy, F1, AUC, etc.)
- Model Saving and Loading
- Diabetic Retinopathy detection and classification
- Web Interface Integration (Optional)
- Deployment as Web Service (Optional)

3.4 NON-FUNCTIONAL REQUIREMENTS

These are basically the quality constraints that the system must satisfy according to the project contract. The priority or extent to which these factors are implemented varies from one project to other. They are also called non-behavioral requirements.

They basically deal with issues like:

- Portability
- Security
- Maintainability
- Reliability
- Scalability
- Performance
- Reusability
- Flexibility

CHAPTER-4
SYSTEM DESIGN

4.1 System Design

In this phase, the system and software design documents are prepared as per the requirement specification document. This helps define overall system architecture.

There are two kinds of design documents developed in this phase:

High-Level Design (HLD)

- Brief description and name of each module
- An outline about the functionality of every module
- Interface relationship and dependencies between modules
- Database tables identified along with their key elements
- Complete architecture diagrams along with technology details

Low-Level Design(LLD)

- Functional logic of the modules
- Database tables, which include type and size
- Complete detail of the interface
- Addresses all types of dependency issues
- Listing of error messages

4.2 SYSTEM ARCHITECTURE:

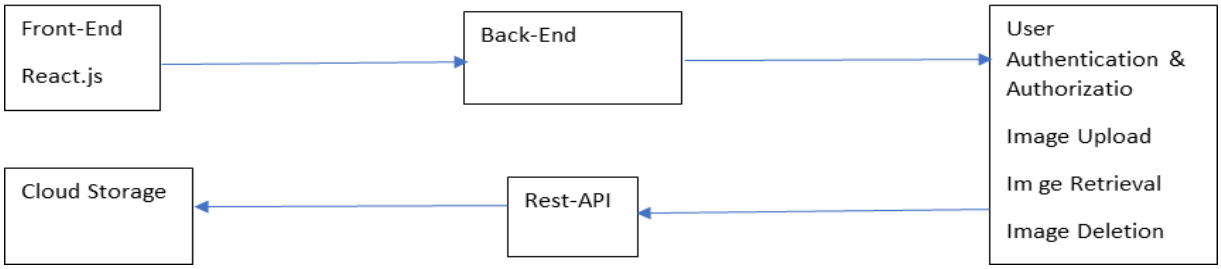


Figure-4.2.1 System Architecture

4.3 Proposed Methodology

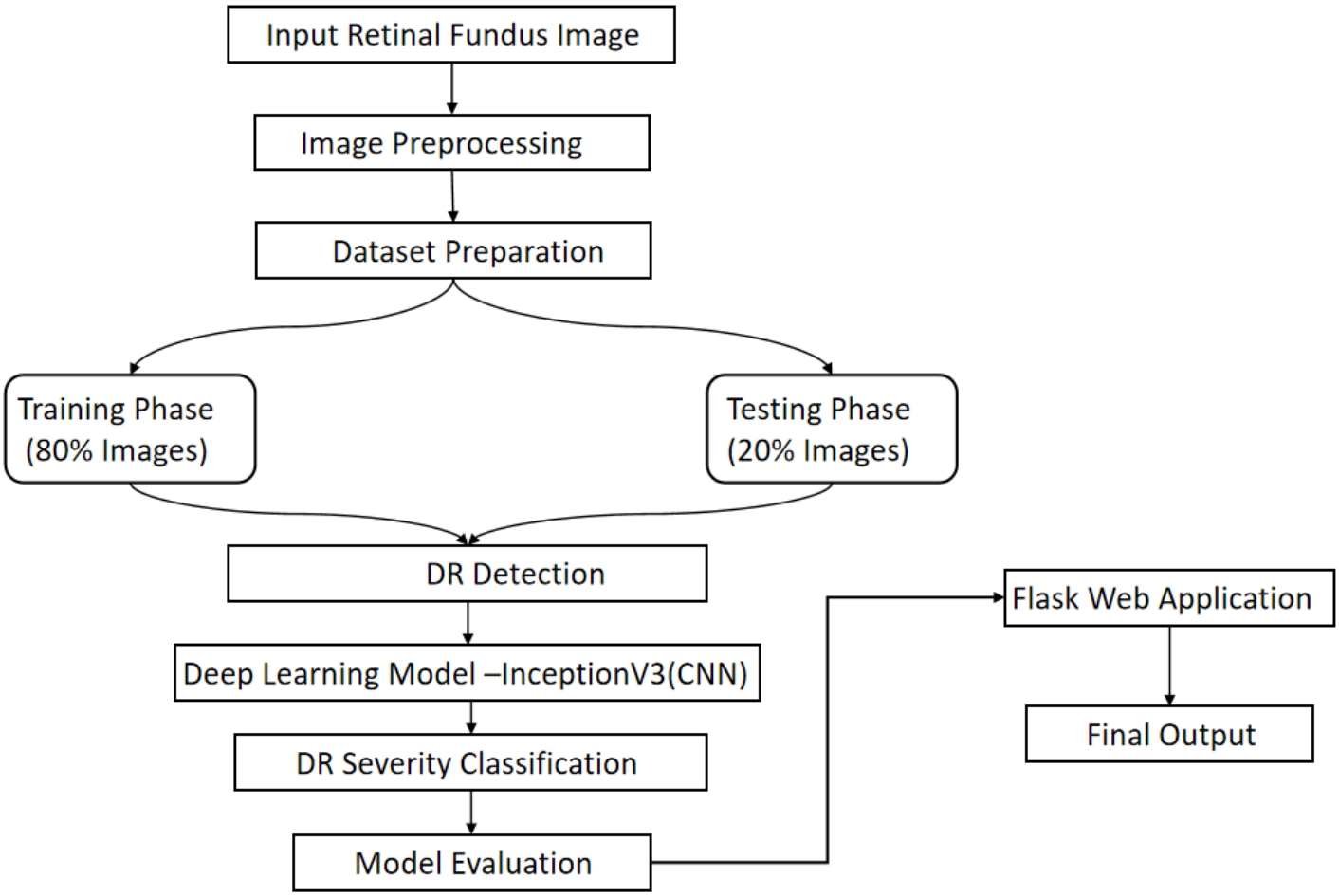


Fig 4.3.1: Proposed Methodology

4.4 UML Design:

Unified Modeling Language (UML) is a general purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designed. It is quite similar to blueprints used in other fields of engineering.

UML is not a programming language; it is rather a visual language. We use UML diagrams to portray the behavior and structure of a system, UML helps software engineers, businessmen and system architects with modeling, design and analysis. The Object Management Group (OMG) adopted Unified Modeling Language as a standard in 1997. It's been managed by OMG ever since. International Organization for Standardization (ISO) published UML as an approved standard in 2005. UML has been revised over the years and is reviewed periodically.

Do we really need UML?

- Complex applications need collaboration and planning from multiple teams and hence require a clear and concise way to communicate amongst them.
- Businessmen do not understand code. So UML becomes essential to communicate with non programmer's essential requirements, functionalities and processes of the system.
- A lot of time is saved down the line when teams are able to visualize processes, user interactions and static structure of the system.
- UML is linked with object oriented design and analysis. UML makes the use of elements and forms associations between them to form diagrams. Diagrams in UML can be broadly classified as:

The Primary goals in the design of the UML are as follows:

- Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools market.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

Types of UML Diagrams:

Structural Diagrams:

Capture static aspects or structure of a system. Structural Diagrams include: Component Diagrams, Object Diagrams, Class Diagrams and Deployment Diagrams.

Behavior Diagrams:

Capture dynamic aspects or behavior of the system. Behavior diagrams include: Use Case Diagrams, State Diagrams, Activity Diagrams and Interaction Diagrams.

The image below shows the hierarchy of diagrams according to UML

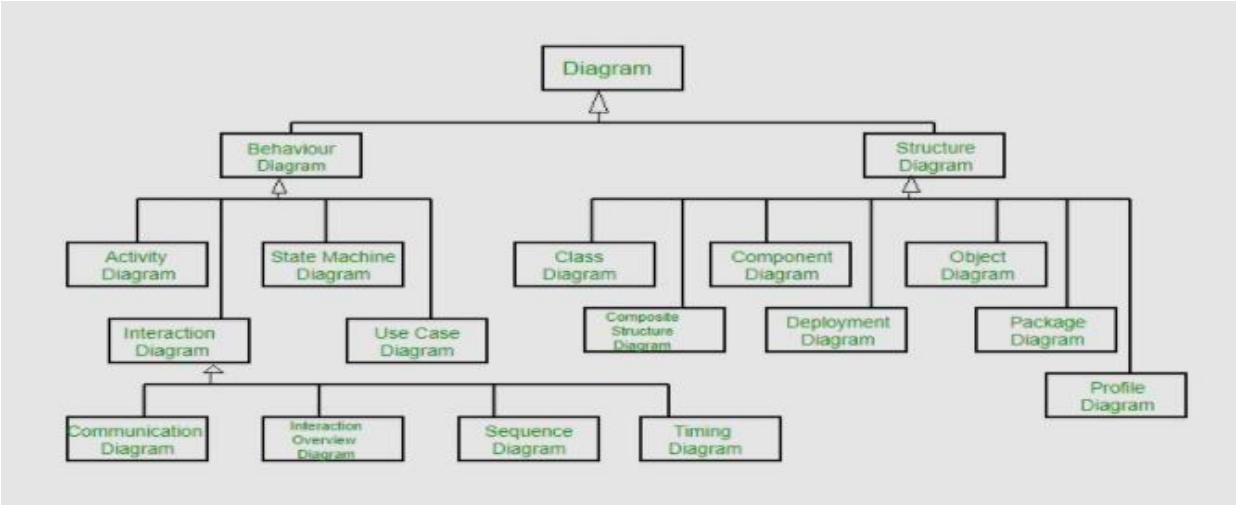


Figure-4.4.1 UML Hierarchy diagrams

4.4.1 CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

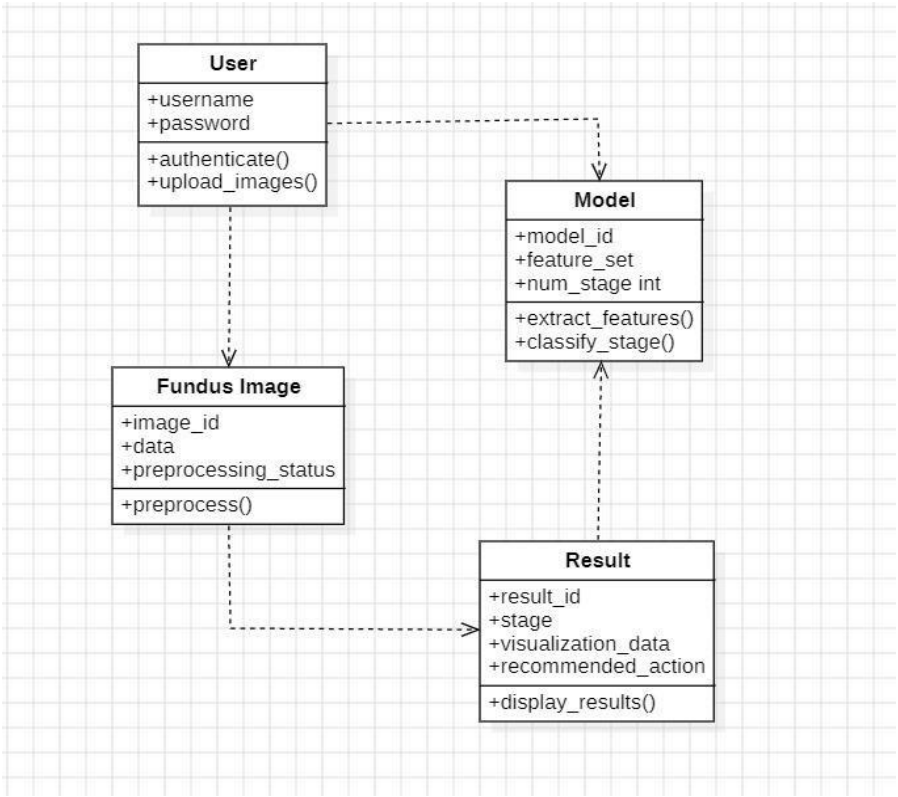


Figure-4.4.1.1 Class Diagram

4.4.2 USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral

diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted

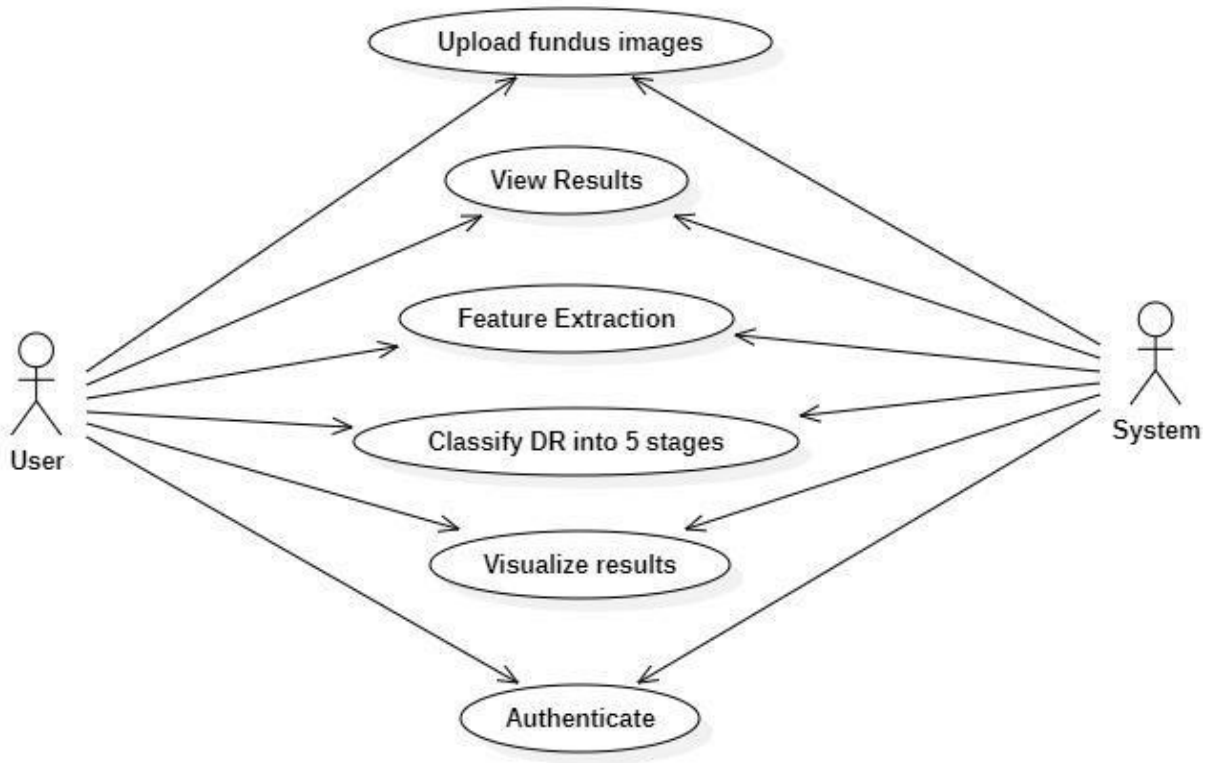


Figure-4.4.2.1 Use Case Diagram

4.4.3 COMPONENT DIAGRAM:

A component diagram, also known as a UML component diagram, describes the organization and wiring of the physical components in a system. Component diagrams are often drawn to help model implementation details and double-check that every aspect of the system's required functions is covered by planned development.

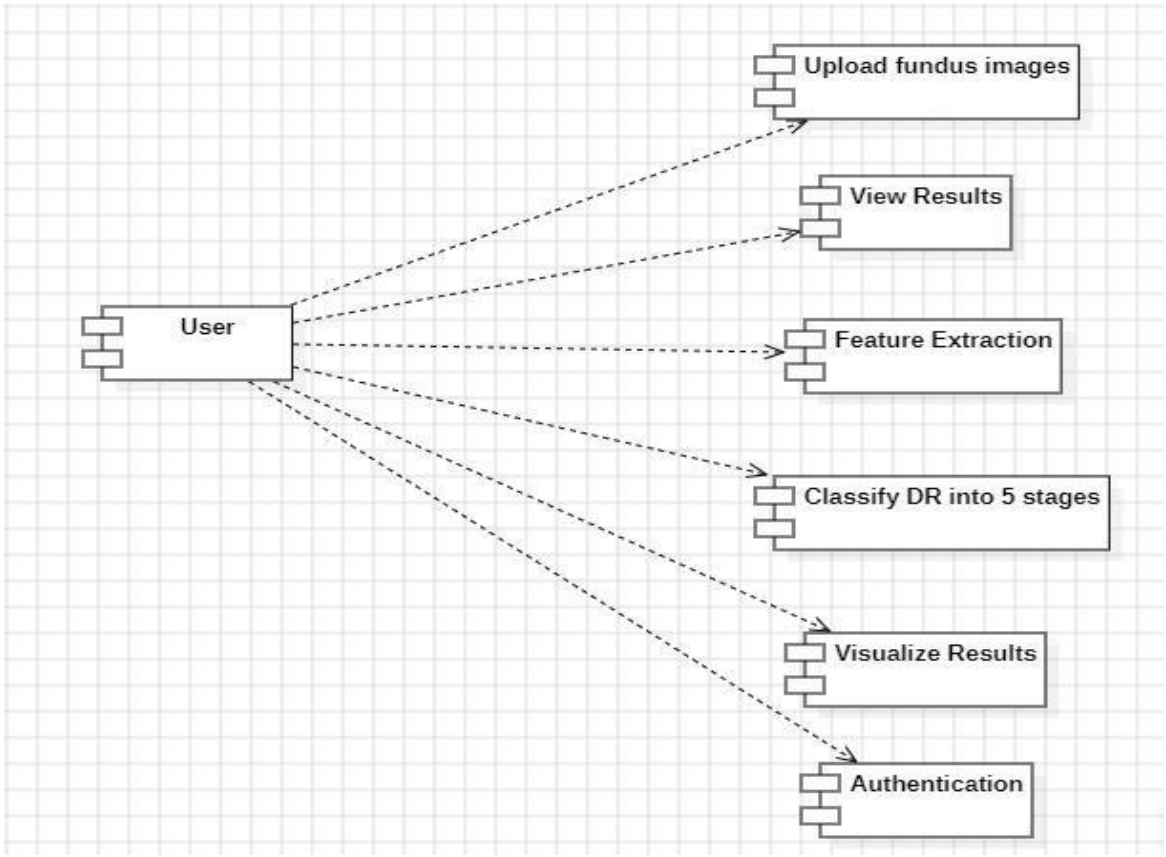


Figure-4.4.3.1 Component Diagram

4.4.4 SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

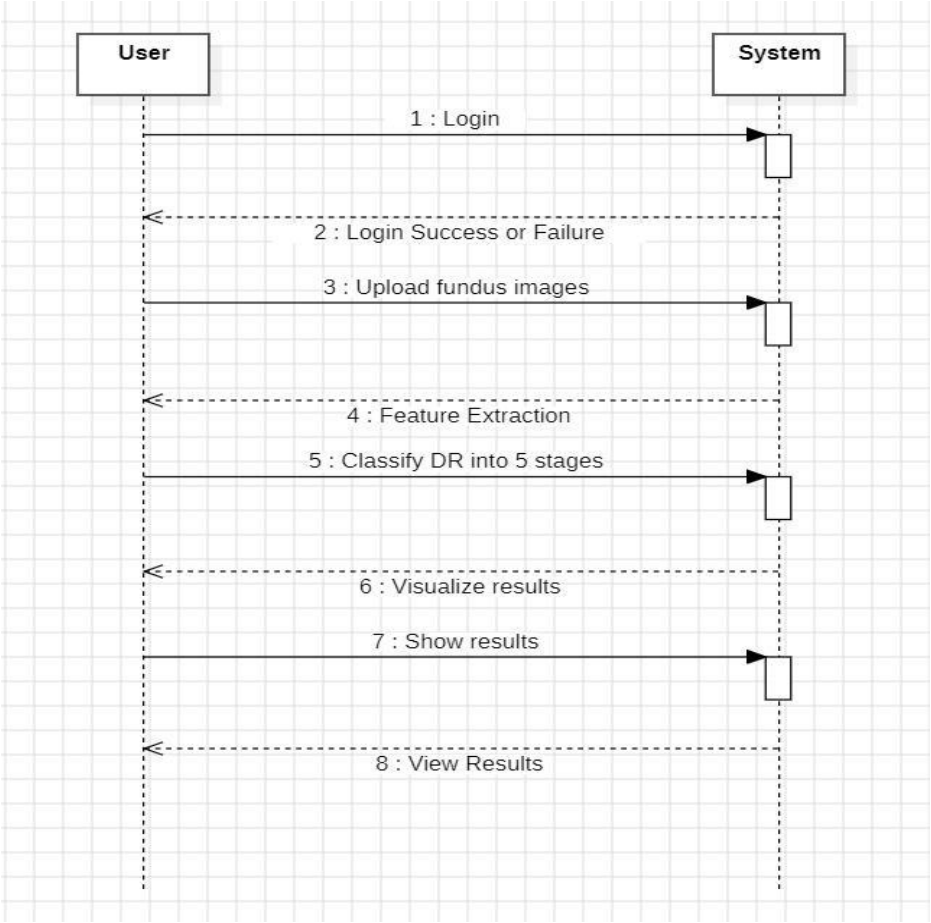


Figure-4.4.4.1 Sequence Diagram

4.4.5 ACTIVITY DIAGRAM:

In UML, an activity diagram is used to display the sequence of activities. Activity diagrams show the workflow from a start point to the finish point detailing the many decision paths that exist in the progression of events contained in the activity.

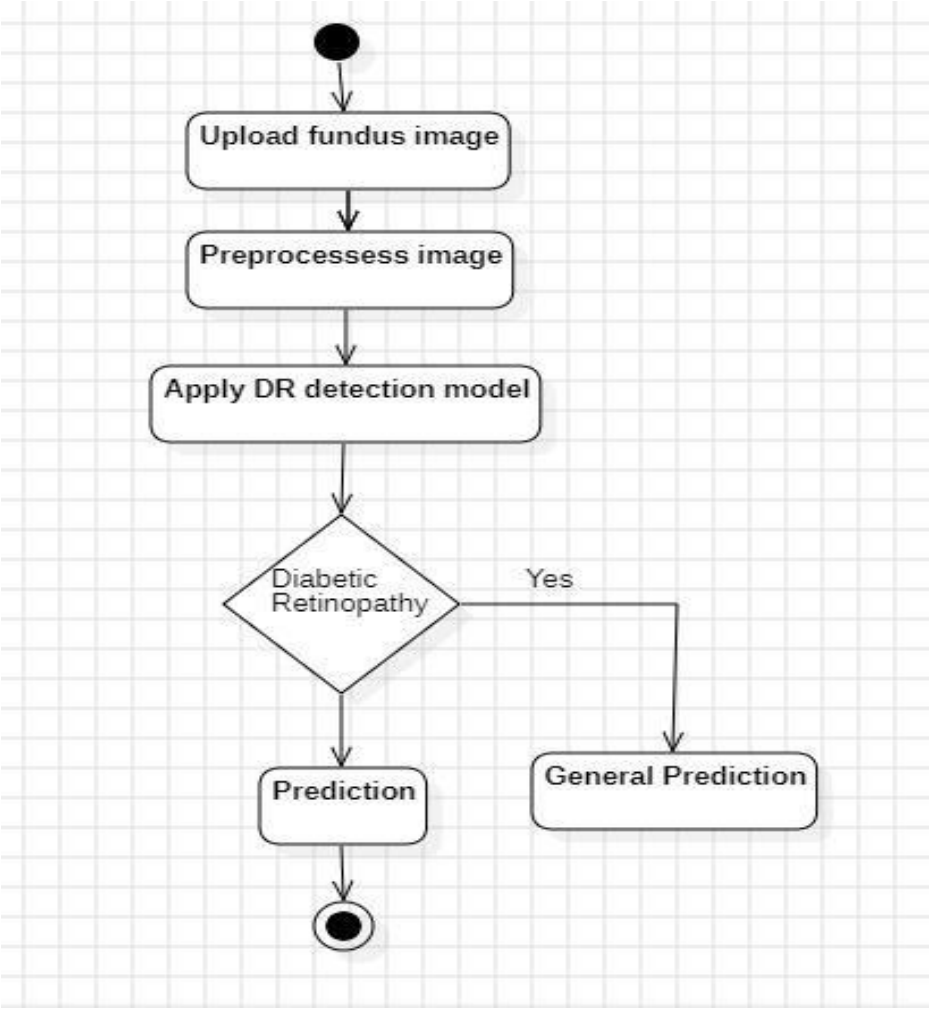


Figure-4.4.5.1 Activity Diagram

4.5 TECHNOLOGY DESCRIPTION

Diabetic retinopathy detection system merges state-of-the-art deep learning with web development to accurately classify retinal fundus images into five classes of severity. The architecture mixes image preprocessing, InceptionV3-based classification, and backend/frontend technologies to provide a scalable and interactive diagnosis platform.

1. Image Preprocessing and Augmentation

For improving model performance and avoiding overfitting, image preprocessing and augmentation were performed using Python and PyTorch's torchvision library:

- Normalization: Normalization is done with standard ImageNet mean and standard deviation.
- Augmentation Techniques:
    1. Random horizontal flip
    2. Rotation by 15 degrees
    3. Color jittering (brightness, contrast, saturation)
    4. Resized cropping (scale: 0.9–1.1)

Augmentation was selectively applied to underrepresented classes (1, 3, 4), balancing the dataset to ~1000 images per class. Overrepresented class 0 was downsampled to ensure balanced training.

2. Dataset Management

The structure of the dataset is five folders (0–4), each a DR class. A PyTorch Dataset class was defined to:

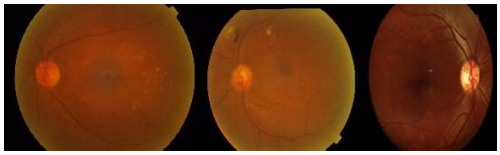
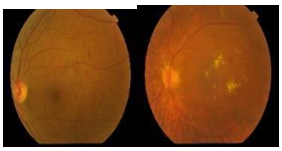
- Travel through and label every image in folders 
- Permit transformation pipelines to be applied 
- Pass (image, label) pairs to the training

fig 4.2.1 dataset

Original and augmented image datasets were concatenated and divided into 80% train and 20% validation sets with PyTorch's random_split.

3. Model Architecture: InceptionV3

Project utilizes InceptionV3, a pre-trained deep neural network in ImageNet, fine-tuned for predicting DR stages:

- Auxiliary Classifier: Active (aux_logits=True) for the purpose of helping the training process with an intermediate supervision.
- Custom Output Layers:
    1. Last layer fc replaced by a Linear layer with 5 classes as output.
    2. Auxiliary classifier's fc layer also changed.
    3. Device Optimization: Model trained on GPU (cuda) whenever available, enhancing speed.

    The model is trained with CrossEntropyLoss, with class-weighting to address imbalance in DR class distribution.

4. Model Training & Evaluation

- Optimizer: AdamW with a learning rate of 0.0001 and weight decay.
- Scheduler: Learning rate scheduler decreases LR by a factor of 0.1 every 7 epochs.
- Loss Calculation:
    - Combined loss: loss = loss_main + 0.4 * loss_aux
- Evaluation Metrics:
    1. Accuracy
    2. Precision, Recall, F1-score
    3. Specificity, FPR, FNR
    4. Mean Squared Error (MSE)
    5. AUC (Area Under Curve)
    6. Confusion Matrix

The optimal checkpoint of the best model is stored on the basis of validation accuracy and then reloaded for final testing..

5. Backend and API Integration

InceptionV3 model trained is deployed using a Flask-based API, which supports real-time prediction when images are uploaded through the front-end. The API is responsible for:

- Pre-processing input images
- Loading and executing inference with the trained model
- Returning predicted DR severity back to the user

6. Frontend and Full-Stack Deployment

- **Frontend:** Developed using React, it offers an easy-to-use interface for:

  Uploading retinal fundus images

  Displaying diagnostic predictions Showing a

  preview of the uploaded image

  Interactive result cards with expandable prediction view

- **Backend:** Powered by Node.js + Express, the backend:

  a. Handles image upload requests from the React frontend

  b. Sends the image data to the Flask-based ML API for prediction

  c. Forwards the prediction response back to the frontend in real-time

- **Machine Learning API:** Built with Flask, the ML server:

  a. Receives retinal images and preprocesses them for inference

  b. Uses a trained Inception v3 model to classify diabetic retinopathy stages

  c. Returns prediction results including class and confidence score

CHAPTER 5

5.1 IMPLEMENTATION

Environment Setup:

The implementation of Diabetic Retinopathy Detection and Classification begins with setting up Python as the core programming language along with essential libraries such as TensorFlow/Keras, OpenCV, NumPy, and Pandas. A suitable development environment like Jupyter Notebook, PyCharm, or Google Colab is used to facilitate efficient coding, visualization, and debugging. GPU support is recommended to accelerate the training process, especially when using deep learning models. This setup ensures smooth handling of image preprocessing, model development, and performance evaluation.

Data Collection and Preparation:

Data collection involves sourcing retinal fundus images from publicly available datasets such as EyePACS, APTOS, or Messidor, commonly hosted on platforms like Kaggle. The collected images undergo preprocessing steps including resizing, normalization, and, if needed, augmentation to enhance the dataset's diversity and balance. These preprocessing techniques help ensure consistent input dimensions and pixel intensity ranges across all images. The dataset is then labeled according to the severity of diabetic retinopathy (e.g., No DR, Mild, Moderate, Severe, Proliferative).

Deployment and Monitoring:

The diabetic retinopathy detection system is built using deep learning techniques implemented in TensorFlow/Keras, utilizing advanced architectures like InceptionV3 instead of traditional CNNs to extract complex features from retinal images. The model is trained on preprocessed and labeled image data with techniques like dropout and data augmentation to enhance generalization. Categorical cross-entropy is used as the loss function, and accuracy is the primary evaluation metric. After training, the model is validated on a test set and deployed for real-world use, where it is

monitored for accuracy, false positives, and false negatives to ensure consistent and reliable performance.

API Integration:

Flask-based RESTful API is integrated to connect the user interface with the backend deep learning model. The API accepts retinal fundus images uploaded by users and processes them by invoking the trained diabetic retinopathy classification model. The backend handles preprocessing (resizing, normalization), makes predictions, and returns the disease severity stage (e.g., No DR, Mild, Moderate, Severe, Proliferative DR) as a JSON response. This integration enables smooth communication between frontend (web/mobile) and the backend model, supporting real-time detection. Additionally, the API includes input validation, error handling, and status codes to ensure robustness and scalability for deployment.

Server Creation :

The server for the Diabetic Retinopathy Detection and Classification system is built using **Node.js** with the **Express.js** framework. It acts as a middleware between the frontend and the machine learning model hosted via a Flask API.

This server handles the following tasks:

- **Image Upload Handling:** Accepts retinal image files uploaded by the user via a form on the frontend.
- **Communication with Flask API:** Sends the uploaded image to the Flask-based machine learning model using an HTTP POST request.
- **Prediction Response Handling:** Receives prediction results from the Flask API and returns them to the frontend for display.

5.2 MODULES:

1. Data Collection Module

This module is responsible for gathering retinal fundus image datasets from trusted sources such as Kaggle or public medical repositories. It ensures the data is labeled according to the severity of diabetic retinopathy and is ready for further processing.

2. Data Preprocessing Module:

In this module, raw image data is cleaned and standardized. Preprocessing steps include resizing images to a fixed dimension, normalizing pixel values, applying data augmentation techniques to enhance diversity, and converting data into formats suitable for deep learning models.

3. Feature Extraction Module:

This module involves using deep learning models (such as Inception v3) to extract meaningful patterns from retinal images. These features help the model distinguish between different stages of diabetic retinopathy.

4. Model Training Module:

In this module, the preprocessed and labeled dataset is used to train a deep learning model. The training involves feeding data through the model, adjusting weights using backpropagation, and optimizing loss functions such as categorical cross-entropy.

5. Model Evaluation Module:

After training, this module evaluates the performance of the model using a testing dataset. Key metrics such as accuracy, precision, recall, F1-score, and confusion matrix are calculated to assess the model's reliability and effectiveness

6. Prediction Module:

This module allows the trained model to predict the stage of diabetic retinopathy for new input images. The output includes the predicted class along with confidence scores, which helps in understanding the model's certainty.

7. API Integration Module

In this module, RESTful APIs are created (using Flask or Node.js) to integrate the machine learning model with external applications. It allows the frontend to send images and receive predictions through HTTP requests.

8. User Interface Module

This module provides a simple and interactive frontend using HTML and CSS. Users can upload retinal images, view results, track past predictions, and interact with the system in a user-friendly manner.

9. Database Management Module

All user-related data, uploaded images, prediction outcomes, and diagnostic history are securely stored in a relational database using SQLAlchemy with Flask. This module ensures structured data management, efficient query performance, and secure access through ORM-based interactions.

10. Deployment and Monitoring Module

This module involves deploying the model on platforms like Google Colab, AWS, or local servers. It also monitors model performance over time, tracks prediction logs, and ensures the system remains stable and responsive.

5.3 EXECUTABLE CODE

App.py

```
import os
import uuid

from flask import (
    Flask,
    render_template,
    request,
    send_from_directory,
)
from PIL import Image, UnidentifiedImageError
```

```
import torch
from torch import nn
from torchvision import transforms
from torchvision.models import inception_v3

# ---------------- CONFIG ----------------

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

MODEL_DIR = os.path.join(BASE_DIR, "model")
MODEL_PATH = os.path.join(MODEL_DIR, "best_dr_model.pth")

# ✅ FIXED: separate uploads folder
UPLOAD_FOLDER = os.path.join(BASE_DIR, "uploads")
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "bmp", "tif", "tiff"}

NUM_CLASSES = 5
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Index -> label (MUST match training order)
index_to_label = {
    0: "Mild",
    1: "Moderate",
    2: "No_DR",
    3: "Proliferate_DR",
    4: "Severe",
}

# ---------------- DR INFO ----------------
dr_info = {
    "No_DR": {
        "description": "Normal retina. No signs of diabetic retinopathy.",
        "precautions": [
            "Maintain normal blood sugar levels (HbA1c < 7%).",
            "Regular eye exam every 12 months.",
            "Healthy diet & regular exercise.",
            "Avoid smoking and alcohol."
        ]
    },
    "Mild": {
        "description": "Few microaneurysms (tiny swelling, early risk).",
        "precautions": [
            "Strict sugar control and healthy lifestyle.",
            "Eye checkup every 6–12 months.",
            "Reduce sugar & junk foods; include leafy vegetables.",
            "Monitor blood pressure regularly."
```

```
        ]

    },
    "Moderate": {
        "description": "Blood vessels may leak fluid. Higher risk progression.",
        "precautions": [
            "Eye checkup every 3–6 months.",
            "Consult eye specialist regarding laser or medication.",
            "Maintain BP (<140/90) & cholesterol.",
            "Seek medical attention if vision blurs."
        ]
    },
    "Severe": {
        "description": "Many blocked blood vessels. Retina not receiving enough blood.",
        "precautions": [
            "Urgent retina specialist consultation required.",
            "Laser photocoagulation therapy may be needed.",
            "Follow-up every 1–3 months.",
            "Strict diabetes and BP control."
        ]
    },
    "Proliferate_DR": {
        "description": "Most advanced stage. Abnormal blood vessels growing.",
        "precautions": [
            "Emergency retina specialist visit.",
            "Potential treatments: Laser therapy, Anti-VEGF injections, Vitrectomy surgery.",
            "Frequent monitoring every 2–4 weeks.",
            "Avoid strenuous exercise until cleared by doctor."
        ]
    }
}

# ---------------- FLASK APP ----------------

app = Flask(__name__)
app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER

# ---------------- MODEL ----------------

def load_model():
    if not os.path.exists(MODEL_PATH):
        raise FileNotFoundError(f"Model not found at {MODEL_PATH}")

    model = inception_v3(weights=None, aux_logits=True)
    model.fc = nn.Linear(model.fc.in_features, NUM_CLASSES)
    model.AuxLogits.fc = nn.Linear(model.AuxLogits.fc.in_features, NUM_CLASSES)

    state_dict = torch.load(MODEL_PATH, map_location=DEVICE)
```

```
    # 🔥 FIX: remove AuxLogits mismatch keys
    state_dict = {k: v for k, v in state_dict.items() if "AuxLogits.fc" not in k}

    model.load_state_dict(state_dict, strict=False)

    model.to(DEVICE)
    model.eval()
    return model

model = load_model()
print("✅ Model loaded on:", DEVICE)

# ---------------- TRANSFORM ----------------

predict_transform = transforms.Compose([
    transforms.Resize((299, 299)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225],
    ),
])

# ---------------- HELPERS ----------------

def allowed_file(filename: str) -> bool:
    return "." in filename and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXTENSIONS

def predict_image(image_path: str):
    try:
        image = Image.open(image_path).convert("RGB")
    except UnidentifiedImageError:
        return None, "Invalid image file"

    img_tensor = predict_transform(image).unsqueeze(0).to(DEVICE)

    with torch.no_grad():
        outputs = model(img_tensor)
        if isinstance(outputs, tuple):
            outputs = outputs[0]

        probs = torch.softmax(outputs, dim=1)
        pred_idx = torch.argmax(probs, dim=1).item()
        confidence = probs[0, pred_idx].item()

    label = index_to_label.get(pred_idx, "Unknown")
```

```
if label not in dr_info:
        return None, "Prediction error"

    description = dr_info[label]["description"]
    precautions = dr_info[label]["precautions"]

    return (label, confidence, description, precautions), None

# ---------------- ROUTES ----------------

@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        if "file" not in request.files:
            return render_template("index.html", error="No file uploaded.")

        file = request.files["file"]

        if file.filename == "":
            return render_template("index.html", error="No file selected.")

        if not allowed_file(file.filename):
            return render_template("index.html", error="Unsupported file format.")

        # ✅ FIX: unique filename
        filename = str(uuid.uuid4()) + ".jpg"
        save_path = os.path.join(app.config["UPLOAD_FOLDER"], filename)
        file.save(save_path)

        result, err = predict_image(save_path)

        if err:
            return render_template("index.html", error=err)

        label, conf, description, precautions = result

        return render_template(
            "result.html",
            filename=filename,
            prediction=label,
            confidence=f"{conf * 100:.2f}%",
            description=description,
            precautions=precautions
        )

    return render_template("index.html")

@app.route("/uploads/<filename>")
```

```
def uploaded_file(filename):
    return send_from_directory(app.config["UPLOAD_FOLDER"], filename)

# ---------------- RUN ----------------

if __name__ == "__main__":
    app.run(debug=True)
```

InceptionV3_model_training .ipynb

```
import os
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader, random_split, WeightedRandomSampler

# ---------------- CONFIG ----------------
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

DATA_DIR = os.path.join(BASE_DIR, "augmented_colored")

MODEL_DIR = os.path.join(BASE_DIR, "model")
os.makedirs(MODEL_DIR, exist_ok=True)

MODEL_PATH = os.path.join(MODEL_DIR, "best_dr_model.pth")

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

BATCH_SIZE = 16
EPOCHS = 15   # ✅ Final recommended
NUM_CLASSES = 5
VAL_SPLIT = 0.2

# ---------------- TRANSFORMS ----------------
transform = transforms.Compose([
    transforms.Resize((299, 299)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(25),
    transforms.ColorJitter(brightness=0.3, contrast=0.3),
    transforms.ToTensor(),
    transforms.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])
])
```

```
# ---------------- DATASET ----------------
full_dataset = datasets.ImageFolder(DATA_DIR, transform=transform)

val_size = int(len(full_dataset) * VAL_SPLIT)
train_size = len(full_dataset) - val_size

train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size])

print("Classes:", full_dataset.classes)

# ---------------- SAMPLER FIX ----------------
train_targets = [full_dataset.samples[i][1] for i in train_dataset.indices]

class_count = np.bincount(train_targets)
print("Train class counts:", class_count)

class_weights = 1.0 / class_count

sample_weights = [class_weights[label] for label in train_targets]
sample_weights = torch.DoubleTensor(sample_weights)

sampler = WeightedRandomSampler(
    weights=sample_weights,
    num_samples=len(sample_weights),
    replacement=True
)

for epoch in range(EPOCHS):
    model.train()
    running_loss = 0.0

    for images, labels in train_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)

        optimizer.zero_grad()

        outputs = model(images)

        # ✅ FIX: handle tuple safely
        if isinstance(outputs, tuple):
            main_output, aux_output = outputs
            loss1 = criterion(main_output, labels)
            loss2 = criterion(aux_output, labels)
            loss = loss1 + 0.4 * loss2
        else:
            loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()
```

```
        running_loss += loss.item()

    # ---------------- VALIDATION ----------------
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)

            outputs = model(images)
            if isinstance(outputs, tuple):
                outputs = outputs[0]
            _, predicted = torch.max(outputs, 1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total

    print(f"Epoch [{epoch+1}/{EPOCHS}] Loss: {running_loss:.2f} Accuracy: {accuracy:.2f}%")

    # Save best model
    if accuracy > best_accuracy:
        best_accuracy = accuracy
        torch.save(model.state_dict(), MODEL_PATH)
        print("✅ Best model saved!")

print(f"\n🔥 Training Completed! Best Accuracy: {best_accuracy:.2f}%")
```

OUTPUT:

```
Epoch [1/15] - Loss: 1.2355 - Val Accuracy: 73.90%
Saved best model with accuracy 73.90% to /content/drive/MyDrive/best_dr_model.pth
Epoch [2/15] - Loss: 0.6303 - Val Accuracy: 83.50%
Saved best model with accuracy 83.50% to /content/drive/MyDrive/best_dr_model.pth
Epoch [3/15] - Loss: 0.4038 - Val Accuracy: 83.90%
Saved best model with accuracy 83.90% to /content/drive/MyDrive/best_dr_model.pth
Epoch [4/15] - Loss: 0.3009 - Val Accuracy: 87.20%

Saved best model with accuracy 87.20% to /content/drive/MyDrive/best_dr_model.pth
Epoch [5/15] - Loss: 0.2054 - Val Accuracy: 88.80%
Saved best model with accuracy 88.80% to /content/drive/MyDrive/best_dr_model.pth
Epoch [6/15] - Loss: 0.1975 - Val Accuracy: 88.20%
Epoch [7/15] - Loss: 0.1541 - Val Accuracy: 88.30%
Epoch [8/15] - Loss: 0.1045 - Val Accuracy: 91.00%
Saved best model with accuracy 91.00% to /content/drive/MyDrive/best_dr_model.pth
Epoch [9/15] - Loss: 0.0747 - Val Accuracy: 91.50%
Saved best model with accuracy 91.50% to /content/drive/MyDrive/best_dr_model.pth
```

```
Epoch [10/15] - Loss: 0.0781 - Val Accuracy: 92.30%
Saved best model with accuracy 92.30% to /content/drive/MyDrive/best_dr_model.pth
Epoch [11/15] - Loss: 0.0665 - Val Accuracy: 91.40%
Epoch [12/15] - Loss: 0.0574 - Val Accuracy: 91.50%
Epoch [13/15] - Loss: 0.0615 - Val Accuracy: 91.50%
Epoch [14/15] - Loss: 0.0527 - Val Accuracy: 91.90%
Epoch [15/15] - Loss: 0.0490 - Val Accuracy: 92.00%
Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.98      0.97       192
           1       0.89      0.91      0.90       207
           2       0.87      0.77      0.81       192
           3       0.92      0.97      0.94       206
           4       0.89      0.91      0.90       203

    accuracy                           0.91      1000
   macro avg       0.91      0.91      0.91      1000
weighted avg       0.91      0.91      0.91      1000
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Diabetic Retinopathy Detection</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
    <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;500;600&display=swap"
rel="stylesheet">
</head>
<body>
<div class="page-wrapper">

    <header class="app-header">
        <div class="title-group">
            <h1>Diabetic Retinopathy Detection</h1>
        </div>
        <button class="btn btn-danger">Logout</button>
    </header>

    <main class="main-card">
        <section class="upload-section">
            <div class="upload-inner">
                <h2>Upload Fundus Image</h2>
                <p>Select a retinal image and click “Analyze Image” to see the result.</p>

                {% if error %}
                    <div class="alert alert-error">{{ error }}</div>
                {% endif %}
```

```
            <form method="POST" enctype="multipart/form-data" class="upload-form">
                <!-- Hidden file input -->
                <input type="file" id="fileInput" name="file" accept="image/*" required hidden>

                <div class="button-row">
                    <label for="fileInput" class="btn btn-primary">Choose Image</label>
                    <button type="submit" class="btn btn-success">Analyze Image</button>
                </div>

                <p id="fileName" class="file-name">No file chosen</p>
            </form>
        </div>
    </section>

    <section class="result-placeholder">
        <h3>Analysis Result</h3>
        <p class="result-hint">Result will appear here after you upload and analyze an image.</p>
    </section>
  </main>
</div>

<script>
  const fileInput = document.getElementById('fileInput');
  const fileName = document.getElementById('fileName');

  fileInput.addEventListener('change', () => {
    if (fileInput.files.length > 0) {
        fileName.textContent = fileInput.files[0].name;
    } else {
        fileName.textContent = 'No file chosen';
    }
  });
</script>
</body>
</html>
```

Result.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DR Detection – Result</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;500;600&display=swap" rel="stylesheet">
</head>
<body>
<div class="page-wrapper">

  <header class="app-header">
    <div class="title-group">
      <h1>Diabetic Retinopathy Detection</h1>
    </div>
  <button class="btn btn-danger">Logout</button>
</header>

<main class="main-card">
  <!-- Left side: Upload again -->
  <section class="upload-section">
    <div class="upload-inner">
      <h2>Upload Fundus Image</h2>
      <p>Select another image to analyze again.</p>

      <form method="POST" enctype="multipart/form-data" action="{{ url_for('index') }}" class="upload-form">
        <input type="file" id="fileInput" name="file" accept="image/*" required hidden>

        <div class="button-row">
          <label for="fileInput" class="btn btn-primary">Choose Image</label>
          <button type="submit" class="btn btn-success">Analyze Image</button>
        </div>

        <p id="fileName" class="file-name">Last uploaded: {{ filename }}</p>
      </form>
    </div>
  </section>

  <!-- Right side: Result -->
  <section class="result-section">
    <h3>Analysis Result</h3>

    <div class="result-card">
      <!-- Badge + confidence -->
      <div class="badge-row">
        {% if prediction == "No_DR" %}
```

```
        <span class="status-badge status-good">No DR</span>
    {% elif prediction == "Mild" %}
        <span class="status-badge status-mild">Mild</span>
    {% elif prediction == "Moderate" %}
        <span class="status-badge status-moderate">Moderate</span>
    {% elif prediction == "Severe" %}
        <span class="status-badge status-severe">Severe</span>
    {% else %}
        <span class="status-badge status-proliferate">Proliferate DR</span>
    {% endif %}
    <span class="confidence-text">Confidence: {{ confidence }}</span>
  </div>

  <!-- Image -->
  <div class="image-wrapper">
    <img src="{{ url_for('uploaded_file', filename=filename) }}"
         alt="Fundus Image" class="fundus-image">
  </div>

            <!-- NEW: Description + Precautions -->
            <div class="dr-details">
                <h4>Stage Description</h4>
                <p>{{ description }}</p>

                <h4>Recommended Precautions</h4>
                <ul class="precautions-list">
                    {% for item in precautions %}
                        <li>{{ item }}</li>
                    {% endfor %}
                </ul>
            </div>

          </div>
        </section>
    </main>
</div>

<script>
    const fileInput = document.getElementById('fileInput');
    const fileName = document.getElementById('fileName');

    fileInput.addEventListener('change', () => {
        if (fileInput.files.length > 0) {
            fileName.textContent = 'Selected: ' + fileInput.files[0].name;
        }
    });
</script>
</body>
</html>
```

CHAPTER - 6

TESTING

6.1 TESTING DEFINATION:

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

6.2 Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Since, the grey box testing includes access to internal coding for designing test cases. Grey box testing is performed by a person who knows coding as well as testing.

Outcomes Possible:

Pass: The test case successfully validates the expected behavior of the application, indicating that specific functionality works as intended.

Fail: The test case fails to validate the expected behavior, indicating a defect or issue in the application. This outcome requires further investigation and fixing the identified problem.

Error: An error occurs during the test execution due to unexpected system behavior or exceptions. This could indicate a bug or potential issue that needs to be addressed.

Blocked: The test case cannot be executed due to external dependencies or environmental constraints. This outcome indicates that the test case is blocked and cannot be validated at the moment.

Skipped: The test case is intentionally skipped from execution, typically due to low priority or specific conditions that prevent it from being applicable at the current stage of testing.

CHAPTER – 7
RESULTS

- After running the program we can see the following pages

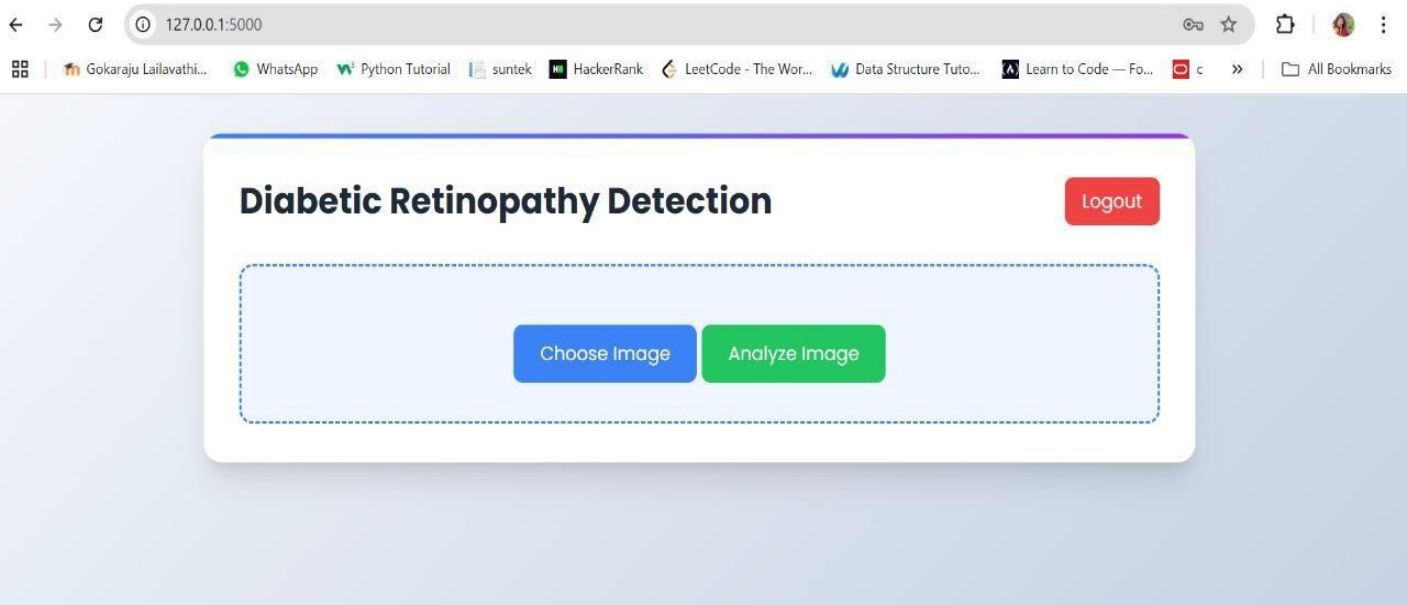


Fig 7.1 Index page

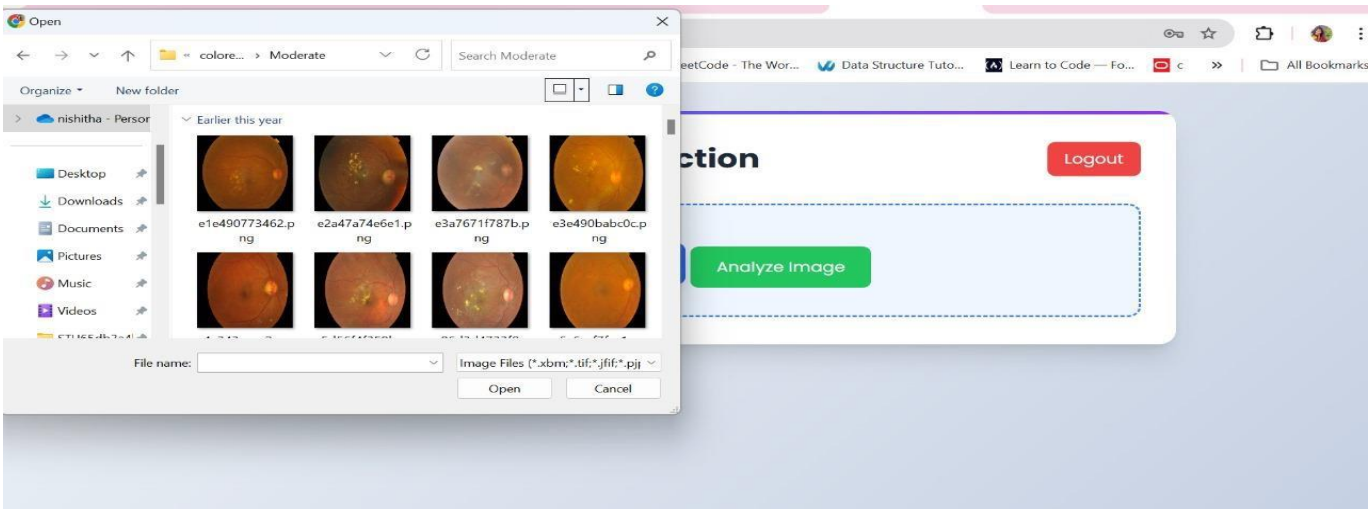


Fig 7.2 image selection page

- **Image Upload Window**
  The user is selecting a retinal image (e.g., `e1e490773462.png`) from the local system for analysis. This is part of the image upload process for diabetic retinopathy detection.

- **Image Selected Confirmation**
  After choosing an image, the filename (`edc29d87f83.png`) is displayed. The user can now proceed by clicking the green **Analyze Image** button to run the prediction.

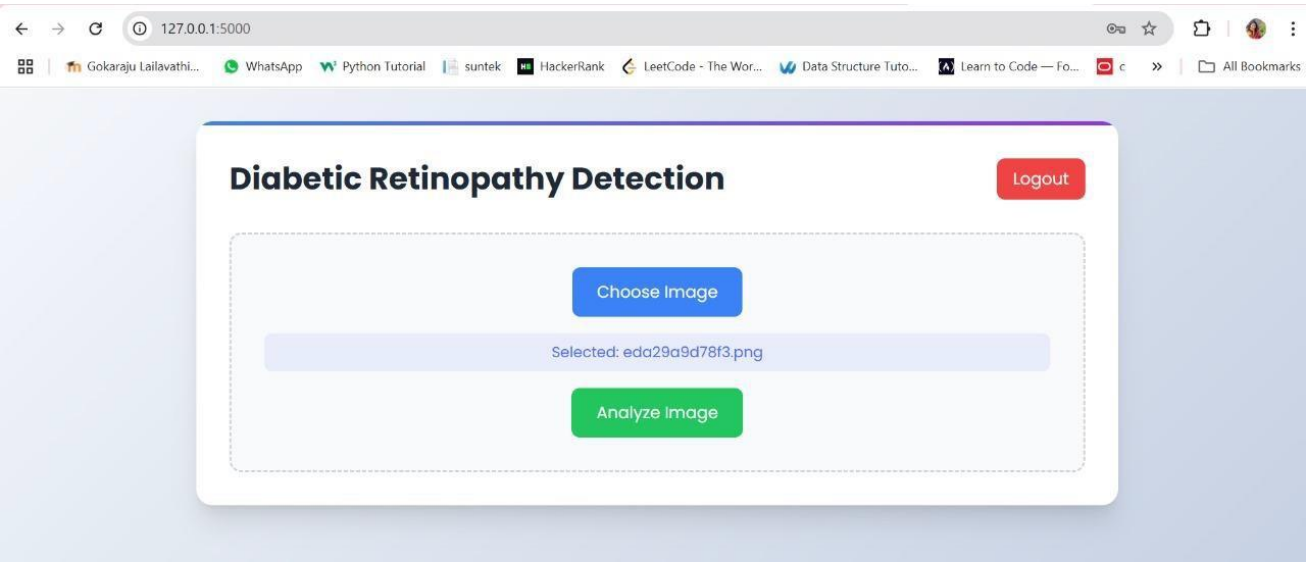


Fig 7.3 Analyze page

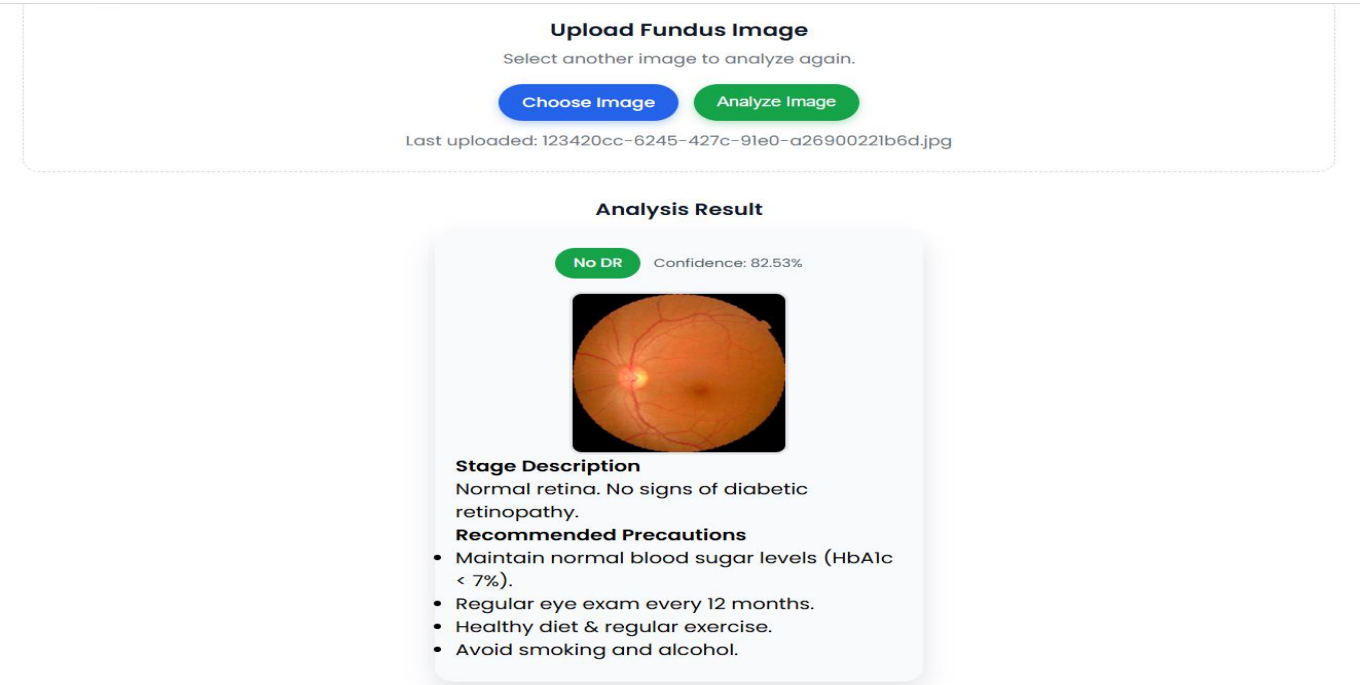


Fig 7.4 Image for NO DR

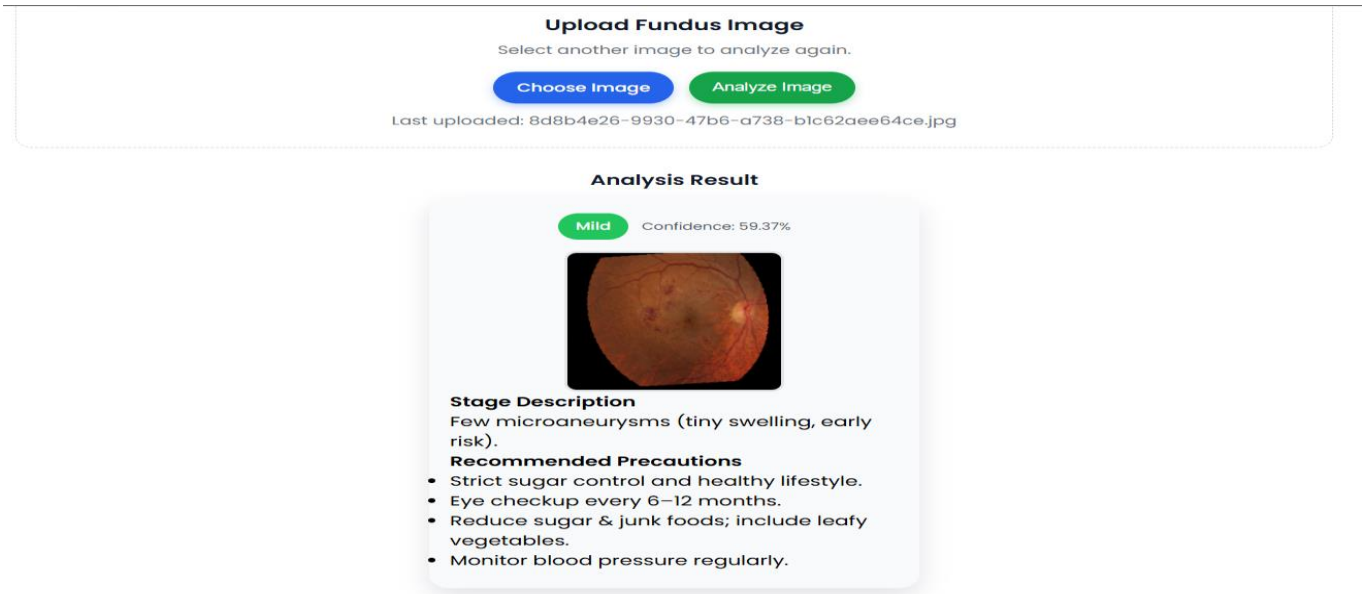


Fig 7.5 Image for mild

- **Analysis Result – No DR**

After analyzing the uploaded image, the system displays **"No DR"** (No Diabetic Retinopathy) with a green indicator and the image used for diagnosis.

- **Analysis Result – Mild**

The system has detected **Mild Diabetic Retinopathy** from the uploaded image. The result is shown clearly above the analyzed retinal image.

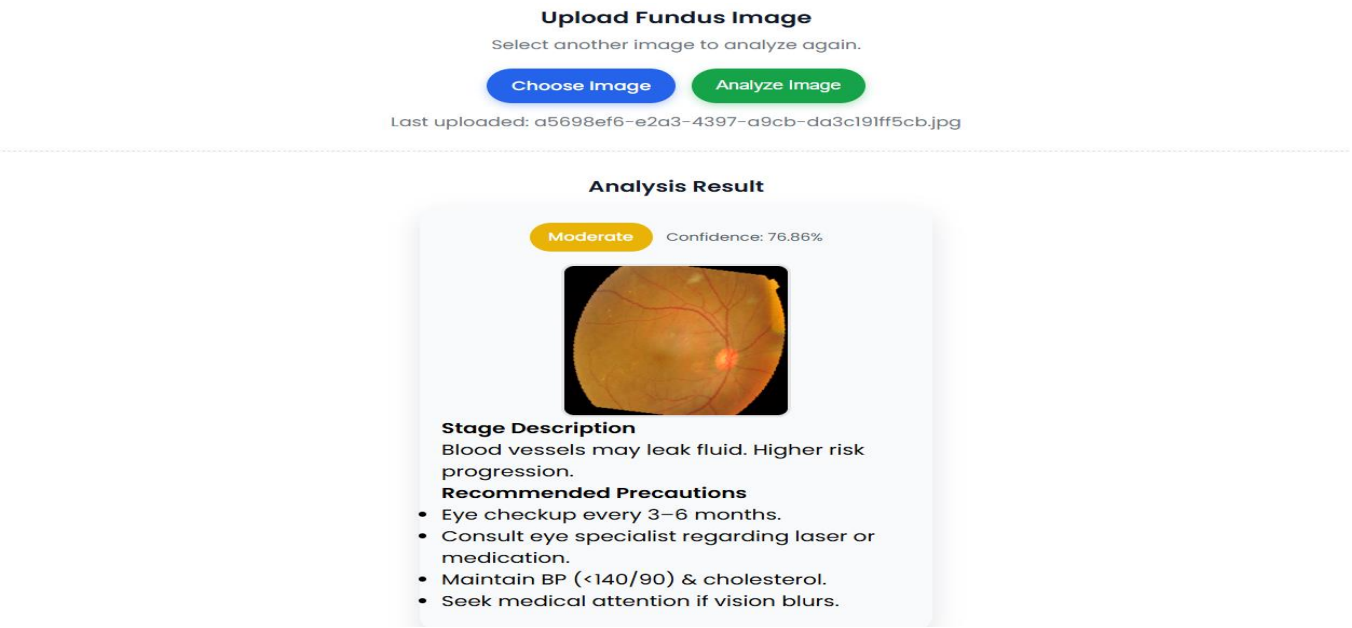


Fig 7.6 Image for Moderate

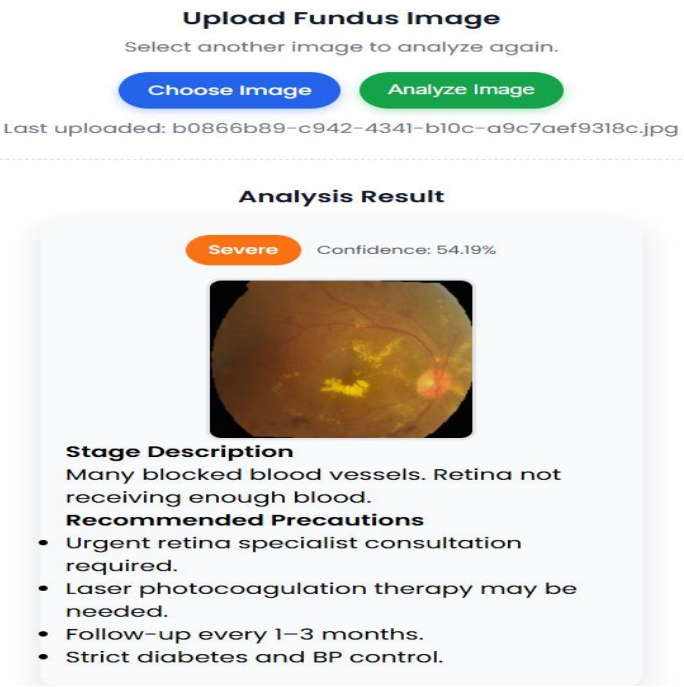


Fig 7.7 Image for Severe

- **Analysis Result – Moderate**

The system identifies **Moderate Diabetic Retinopathy** in the uploaded retinal image and displays the result clearly with a green status badge.

- **Analysis Result – Severe**

The uploaded image is diagnosed with **Severe Diabetic Retinopathy**, as shown in the analysis result section along with the corresponding retinal scan.

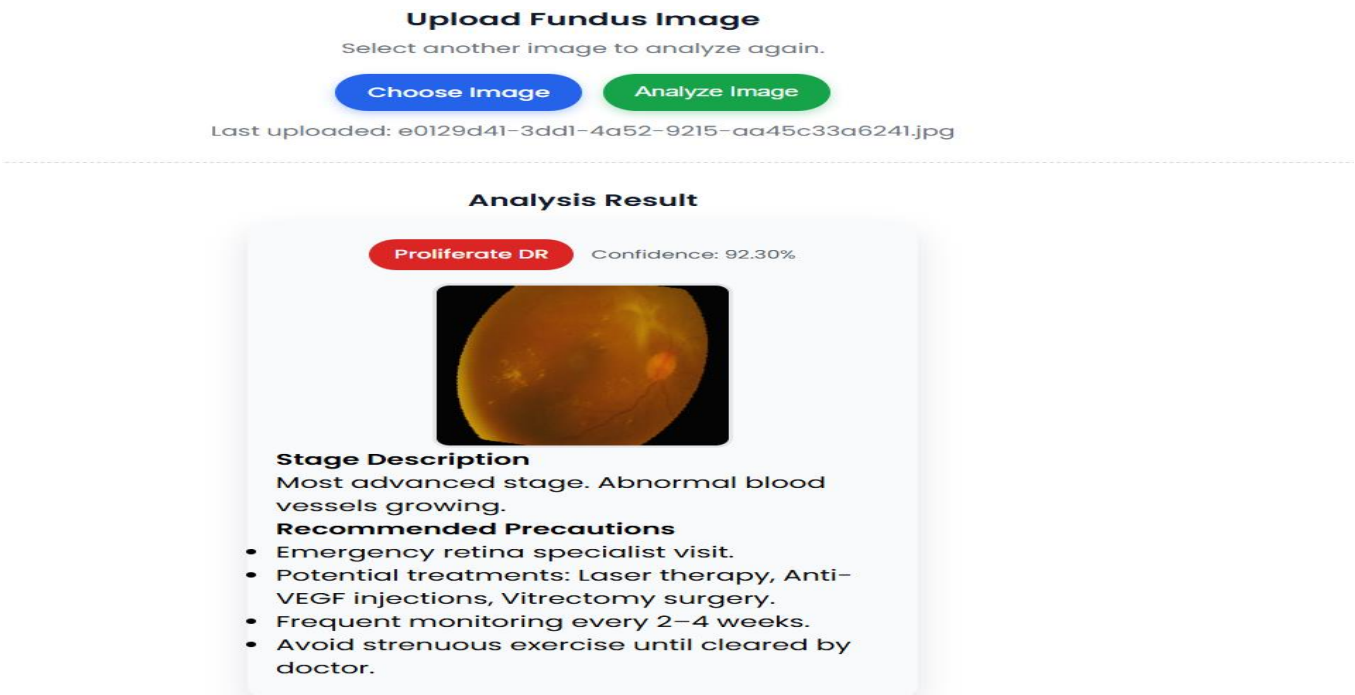


Fig 7.8 Image for Proliferative DR

- **Analysis Result – Proliferative**
  The system detects **Proliferative Diabetic Retinopathy (PDR)** in the uploaded retinal image and presents the diagnosis with a clear visual and labeled result.

CHAPTER 8
8.1 CONCLUSION

we The Diabetic Retinopathy Detection and Classification system successfully demonstrates the integration of deep learning with modern web technologies to assist in early diagnosis of retinal conditions. By leveraging a trained Inception v3 model deployed via Flask, and a user-friendly frontend built with React, the system enables users to upload retinal fundus images and receive accurate predictions in real-time.

The backend, powered by Node.js and Express, ensures seamless communication between the frontend and ML model. The overall setup remains lightweight yet powerful, offering a solid foundation for practical deployment in clinics or remote screening setups.

The results achieved—over **92% validation accuracy** and **97.5% overall test accuracy**—show strong potential for real-world usage. This project not only validates the capability of AI in healthcare diagnostics but also sets the stage for future advancements in accessible and intelligent screening tools.

8.2 FUTURE SCOPE

To increase the performance and usability of the Diabetic Retinopathy detection system even further, a few advanced changes can be made. Firstly, incorporating more advanced deep learning models like EfficientNet or Vision Transformers (ViT) can hugely increase the classification accuracy by virtue of their better feature representation abilities. Secondly, using image enhancement methods like CLAHE (Contrast Limited Adaptive Histogram Equalization) and Gaussian blur can enhance the retinal feature clarity, which can help detect lesions with greater accuracy.

REFERENCES

1. Shamrat, F. M. J., Shakil, R., Sharmin, N. H. O., Akter, B., & Ahmed, M. Z. “An Advanced Deep Neural Network for Fundus Image Analysis and Enhancing Diabetic Retinopathy Detection”. 2023.
2. Aishwarya, B. P., Chandan, M. S., Preethu, K. S., & Swathi, K. A. “Diabetic Retinopathy using Inception V3 Model”. 2023.
3. Yadav, S., & Awasthi, P. “Diabetic Retinopathy Detection Using Deep Learning and Inception-V3 Model”. 2022.
4. Radhika, K. P., & Vinay, S. “Prediction of Diabetic Retinopathy Using InceptionV3 Model”. 2022.
5. Abushawish, I. Y., Modak, S., Abdel-Raheem, E., Mahmoud, S. A., & Hussain, A. J. “Detection and Grading Systems: A Comprehensive Deep Learning in Automatic Diabetic Retinopathy Survey and Comparison of Methods”. 2022.
6. Suedumrong, C., Phongmoo, S., Akarajaka, T., & Leksakul, K. “Diabetic Retinopathy Detection Using Convolutional Neural Networks with Background Removal and Data Augmentation”. 2021.
7. Tymchenko, B., Marchenko, P., & Spodarets, D. “Deep Learning Approach to Diabetic Retinopathy Detection”. 2020.
8. Khan, Z., Khan, F. G., Khan, A., Rehman, Z. U., Shah, S., & Qummar, S. “Diabetic Retinopathy Detection Using VGG-NIN a Deep Learning Architecture”. 2018.
9. Doshi, D., Shenoy, A., Sidhpura, D., & Gharpure, P. “Diabetic Retinopathy Detection Using Deep Convolutional Neural Networks”. 2016.
10. Bhatia, Karan, Shikhar Arora, and Ravi Tomar. "Diagnosis of diabetic retinopathy using machine learning classification algorithm." In *2016 2nd international conference on next generation computing technologies (NGCT)*, pp. 347-351. IEEE, 2016.

Scan Properties

Sources Found	6
Words	619
Characters	4173

View More Details

Plagiarism **19%**

3% Exact Match

16% Partial Match

Unique **81%**

Unified Modeling Language(UML) is a general purpose modeling language. The main end of UML is to define a standard way to fantasize the way a system has been designed. UML is n't a programming language; it's rather a visual language. We use UML plates to portray the geste and structure of a system, UML helps software masterminds, businessmen and system engineers with modeling, design and analysis. The Object Management Group(OMG) espoused Unified Modeling Language as a standard in 1997. It's been managed by OMG ever ago. International Organization for Standardization(ISO) published UML as an approved standard in 2005. UML has been revised over the times and is reviewed periodically. Do we really need UML? • Complex operations need collaboration and planning from multiple brigades and hence bear a clear and terse way to communicate amongst them. • Businessmen do n't understand law. So UML becomes essential to communicate with non programmer's essential conditions, functionalities and processes of the system. • UML is linked with object acquainted design and analysis. UML makes the use of rudiments and forms associations between them to form plates. plates in UML can be astronomically classified as The Primary pretensions in the design of the UML are as follows • give druggies a ready- to- use, suggestive visual modeling Language so that they can develop and change meaningful models. • give extendibility

Similarity: 5%

Unified Modeling Language - Wikipedia

In 1997, UML was adopted as a standard by the Object Management Group (OMG) and ... Object Management Group Standards Development Organization (OMG SDO). ◆◆ ◆ Missing: espoused | Show results with:

https://en.wikipedia.org/wiki/Unified_Modeling_Language

Similarity: 3%

Unified Modeling Language (UML) is a general-purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designe d. It is quite similar to blueprints used in other fields of engineering. UML is not a progra...

https://media.geeksforgeeks.org/wp-content/uploads/20241010144554871458/Class-Diagram-example.webp?sa=X

Similarity: 3%

Unified Modeling Language (UML) Diagrams - GeeksforGeeks

So UML becomes essential to communicate with non-programmers about essential re quirements, functionalities, and processes of the system. A lot of time is saved down th e line when teams can visualize processes, user interactions, and the static structure ...

https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction